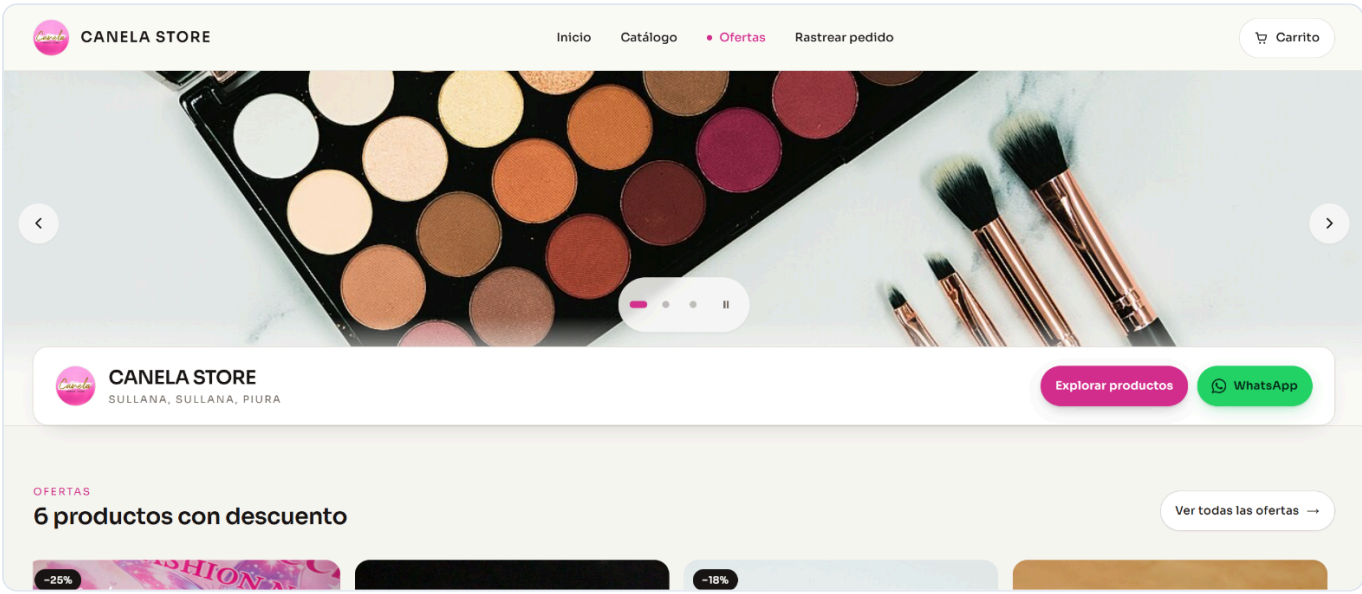
CASO DE ESTUDIO · COMERCIO ELECTRÓNICO PARA MYPES · 2026

Canela Store: tienda online y punto de venta sobre una plataforma multi-tienda

Plataforma tipo «Kyte» hecha a medida: catálogo público con variantes, carrito y checkout sin crear cuenta, pagos por Yape o Plin con confirmación, punto de venta en mostrador, inventario que baja solo y reportes de utilidad. En producción en **tiendas.bjota.online**.

Next.js 15 · Express · TypeScript · PostgreSQL 16 · Prisma · Row Level Security · Docker · Vitest



AUTOR
Bryan Jeremy Espinoza Herrera
BJOTA Systems · Perú

CLIENTA PILOTO
Canela Store
maquillaje y regalos · Sullana, Piura

DESARROLLO
Ago. – oct. 2026
366 commits

01 Resumen

Canela Store es una tienda de maquillaje y regalos que vendía por WhatsApp e Instagram y llevaba el stock en una app de terceros que no lo descontaba sola cuando entraba un pedido. Construí una **plataforma multi-tienda** donde Canela tiene su propia tienda online y un panel para vender en mostrador, confirmar pagos, controlar inventario y ver cuánto ganó. La dueña vio la demo y pidió seis mejoras; las seis se implementaron.

5,600+ pruebas automáticas (API y web)	**83** migraciones de base de datos	**366** commits en 7 semanas	**6/6** pedidos de la clienta implementados

02 El problema

Ventas solo por chat
Cada venta era una conversación: fotos, precios y datos de envío a mano. Sin catálogo propio para compartir.

Stock que no se actualizaba
La app que usaba no descontaba el stock al entrar un pedido online: se vendía lo que ya no había.

Pagos por Yape difíciles de controlar
Constancias sueltas en el chat, sin saber qué pedido estaba pagado y cuál no.

No se sabía cuánto se ganaba
Sin costo por producto ni gastos registrados, no había utilidad real por mes.

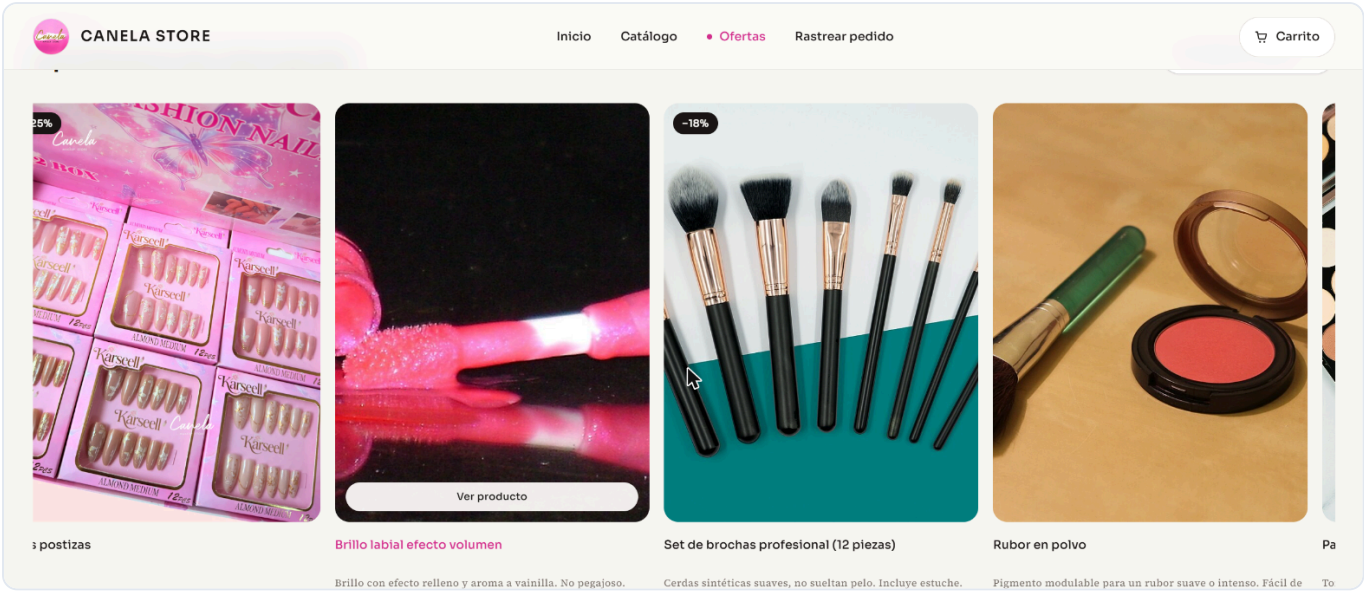
03 La solución

Tienda online propia
Catálogo con variantes de tono, ofertas, carrito y checkout en 3 pasos. Seguimiento del pedido sin cuenta.

Pedidos y pagos ordenados
El cliente adjunta su constancia de Yape o Plin; la dueña la confirma y el pedido avanza por estados.

Mostrador, stock y utilidad
Punto de venta para la tienda física, inventario con kardex que baja solo y reportes de margen y utilidad.

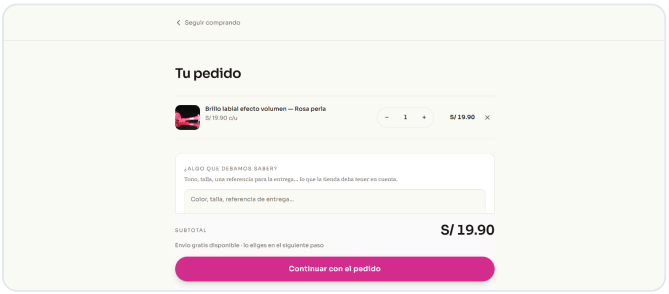
04 La tienda online (en producción)



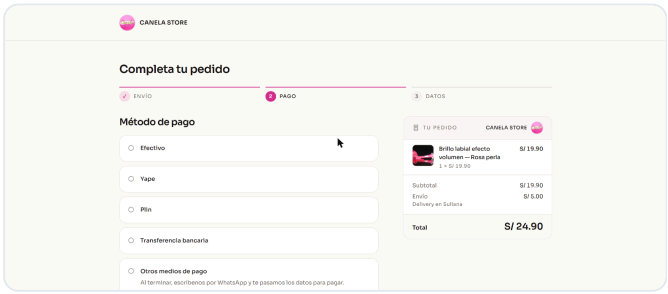
Ofertas con descuento automático en el precio.



Ficha de producto con galería y variantes de tono.

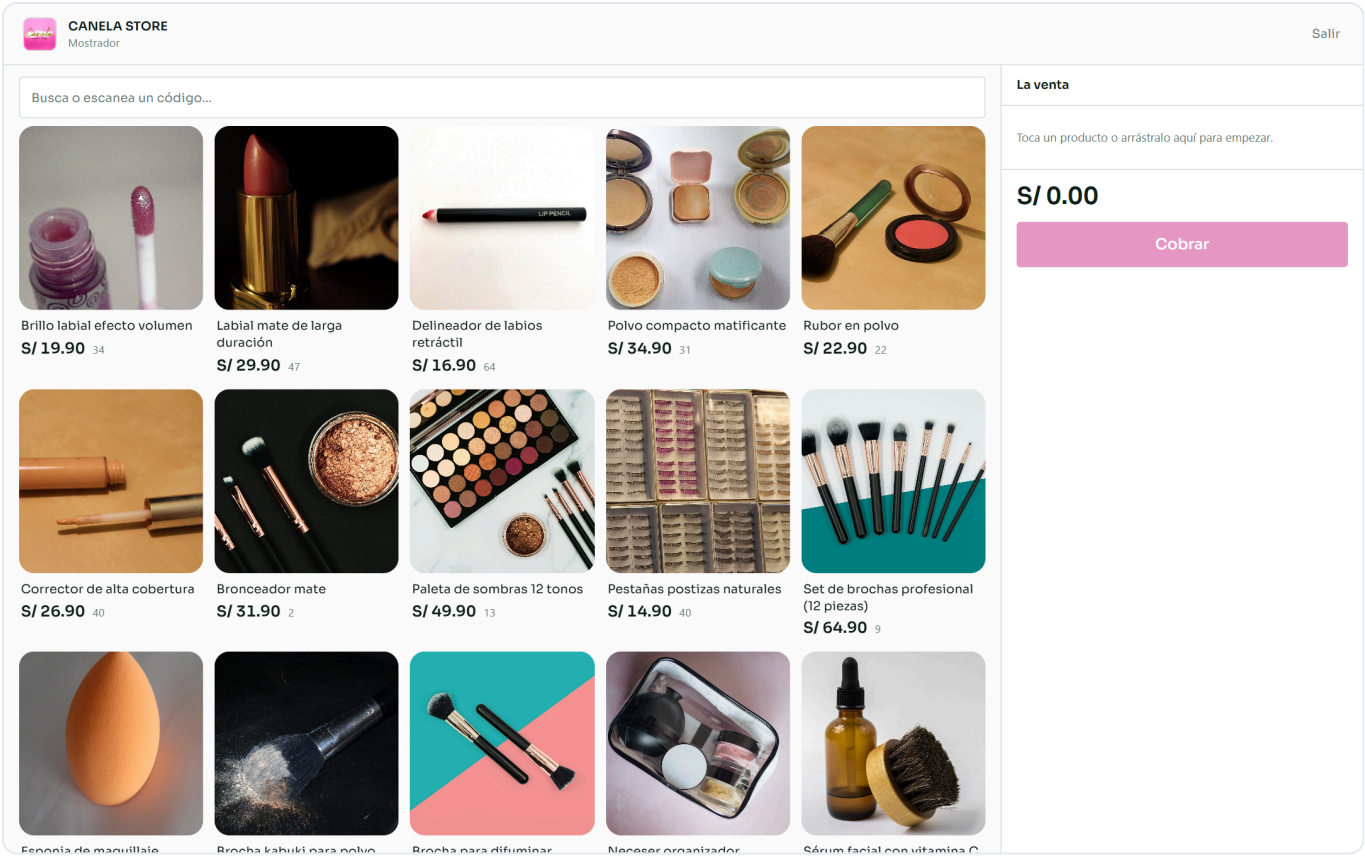


Carrito guardado en el navegador del cliente.

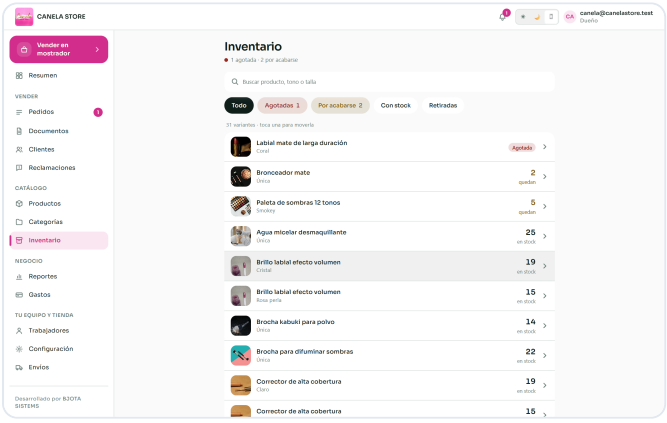


Checkout: envío, pago (efectivo, Yape, Plin, transferencia u otros) y datos.

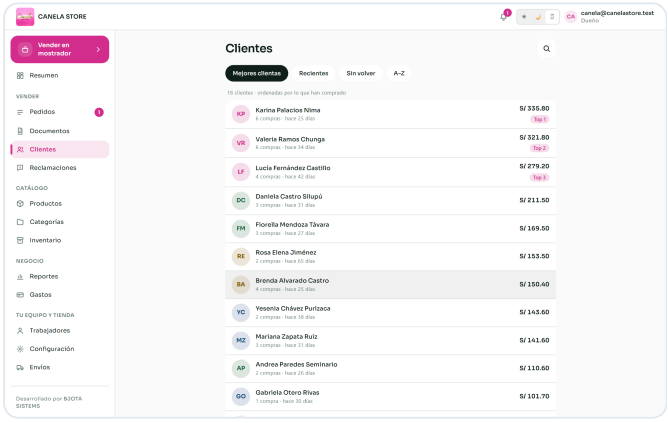
05 El panel de la dueña



Mostrador (POS): vender en la tienda física con pago dividido y vuelto; el stock baja al instante.

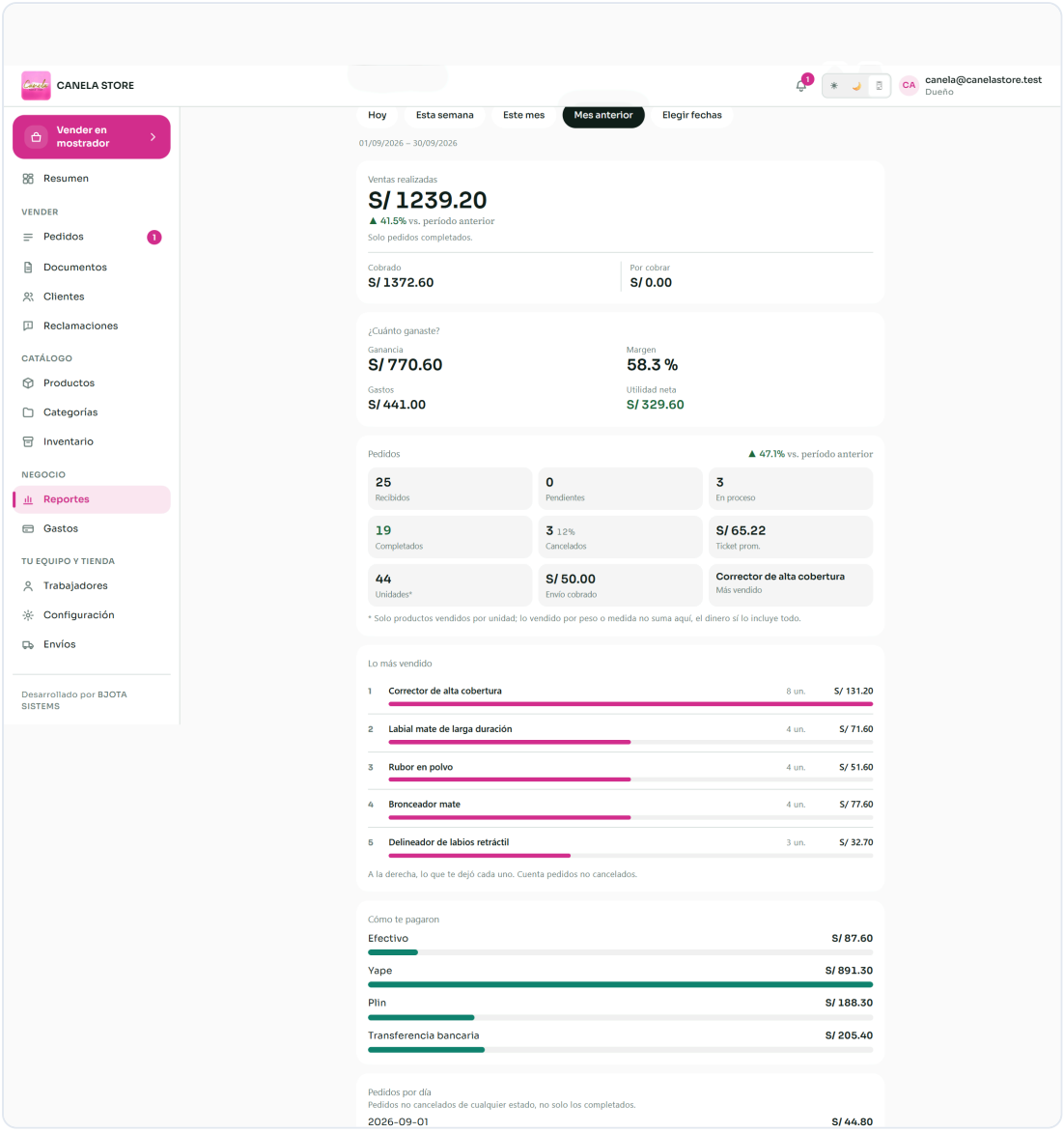


Inventario con alertas de agotado y por acabarse.



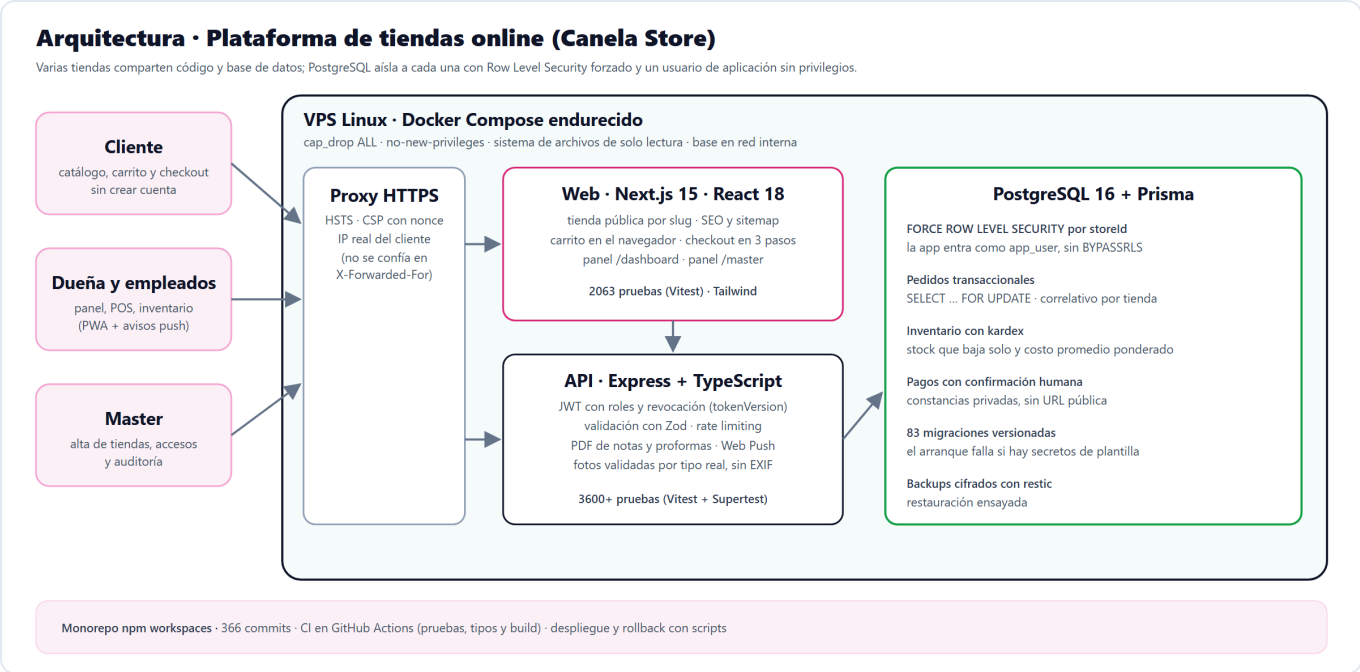
Clientes ordenados por lo que más compran.

06 Reportes: cuánto se ganó



Reporte de septiembre (datos de demostración): ventas, ganancia y margen, utilidad neta tras gastos, productos más vendidos y cobros por método.

07 Arquitectura



Decisiones clave

Decisión	Por qué
RLS forzado + usuario sin privilegios	Cada fila lleva `storeId` y PostgreSQL filtra con `FORCE ROW LEVEL SECURITY`. La app se conecta como `app_user` sin `BYPASSRLS`: un fallo en la API no expone datos de otra tienda.
Pedidos transaccionales	Checkout, venta en mostrador y proforma pasan por el mismo núcleo con `SELECT ... FOR UPDATE`: dos clientes no pueden comprar la última unidad a la vez.
Confirmación humana del pago	Sin pasarela: el cliente declara su pago y adjunta la constancia, que se guarda en almacenamiento privado sin URL pública. Solo quien tiene el permiso `payments.confirm` la confirma.
Sesiones revocables	JWT con `tokenVersion` para cerrar sesiones a distancia; acceso por enlaces de un solo uso.
Secretos que no se olvidan	El servidor se niega a arrancar si detecta secretos de plantilla sin cambiar.
Contenedores endurecidos	`cap_drop: ALL`, `no-new-privileges`, sistema de archivos de solo lectura y base de datos en una red interna sin salida.
Notas de venta, no boletas	Los PDF son notas internas y proformas; no se presentan como comprobantes SUNAT.

08 Seguridad probada

- **Auditoría de seguridad** de la Fase 1, sin hallazgos críticos ni altos.
- **Pruebas de ataque en un entorno réplica:** nombres de tienda con `<script>alert(1)</script>` para comprobar que no hay XSS, cabecera `X-Forwarded-For` falsificada contra el rate limiting, HTTPS, HSTS y CSP.
- **Rate limiting** en el login y en la creación pública de pedidos; "olvidé mi contraseña" endurecido.
- **Imágenes:** se valida el tipo real del archivo y se eliminan los metadatos EXIF (ubicación del celular).
- **Backups cifrados** con restic y un ensayo de restauración documentado.
- **CI** en GitHub Actions: pruebas, chequeo de tipos y build en cada cambio.

09 Lo que pidió la clienta

Tras ver la demo, la dueña de Canela pidió seis cambios. Todos quedaron implementados:

- Modo mostrador (POS) para vender en la tienda física.
- Checkout que se adapta al tipo de envío (recojo, delivery local, agencia a todo el Perú).
- Método de pago "Otros / a coordinar" por WhatsApp.
- Editar pedidos ya creados.
- Nota de venta en PDF.
- Precio por mayor según la cantidad.

10 Resultados

- **En producción** en `tiendas.bjota.online`, con la tienda de Canela cargada como demo para la clienta.
- **Calidad:** 2063 pruebas de la web verificadas el 17/10/2026 y 3622 de la API en la última corrida completa registrada (más 7 omitidas).
- **Pendientes declarados:** correo transaccional, backups fuera del servidor, pasarela de pago y facturación electrónica SUNAT.

11 Lo que aprendí

- **Escuchar a la clienta cambia el producto.** Lo que más valoró (vender en mostrador y checkout según envío) no estaba en el plan inicial.
- **Concurrencia y dinero.** Bloquear filas y usar transacciones evita vender lo que no hay; es invisible para el usuario y crítico para el negocio.
- **Atacar mi propio sistema.** Probar XSS, cabeceras falsificadas y restauración de backups antes de que lo haga otro es lo que me interesa de la ciberseguridad.
- **IA como herramienta con método.** Desarrollé con asistentes de IA trabajando con planes, pruebas y checkpoints verificados.

12 Mi rol

Proyecto propio completo: levanté requisitos con la dueña de Canela, diseñé la base de datos y la arquitectura, desarrollé la API y la web con apoyo de asistentes de IA, escribí las pruebas, endurecí y desplegué la infraestructura, e implementé los cambios que pidió la clienta.

La tienda pública se capturó en producción sin enviar pedidos. El panel proviene de la base de desarrollo local con datos de demostración; fotos de productos de Wikimedia Commons con licencia libre. No se publican el código fuente, datos de clientes reales ni detalles internos del servidor.

Bryan Jeremy Espinoza Herrera · BJOTA Systems · 2026